

EventConductor demo

Arquitectura del montaje

Reservas gobernadas por workflow · IA con control plane propio

GitOps · identidad federada · observabilidad

`ec1.mateu.io` · `console.ec1.mateu.io`

La idea en una frase

El **negocio corre** en un plano de datos gobernado por workflows;
la **IA y la configuración se gobiernan** en un plano de control aparte,
detrás de su propia identidad.

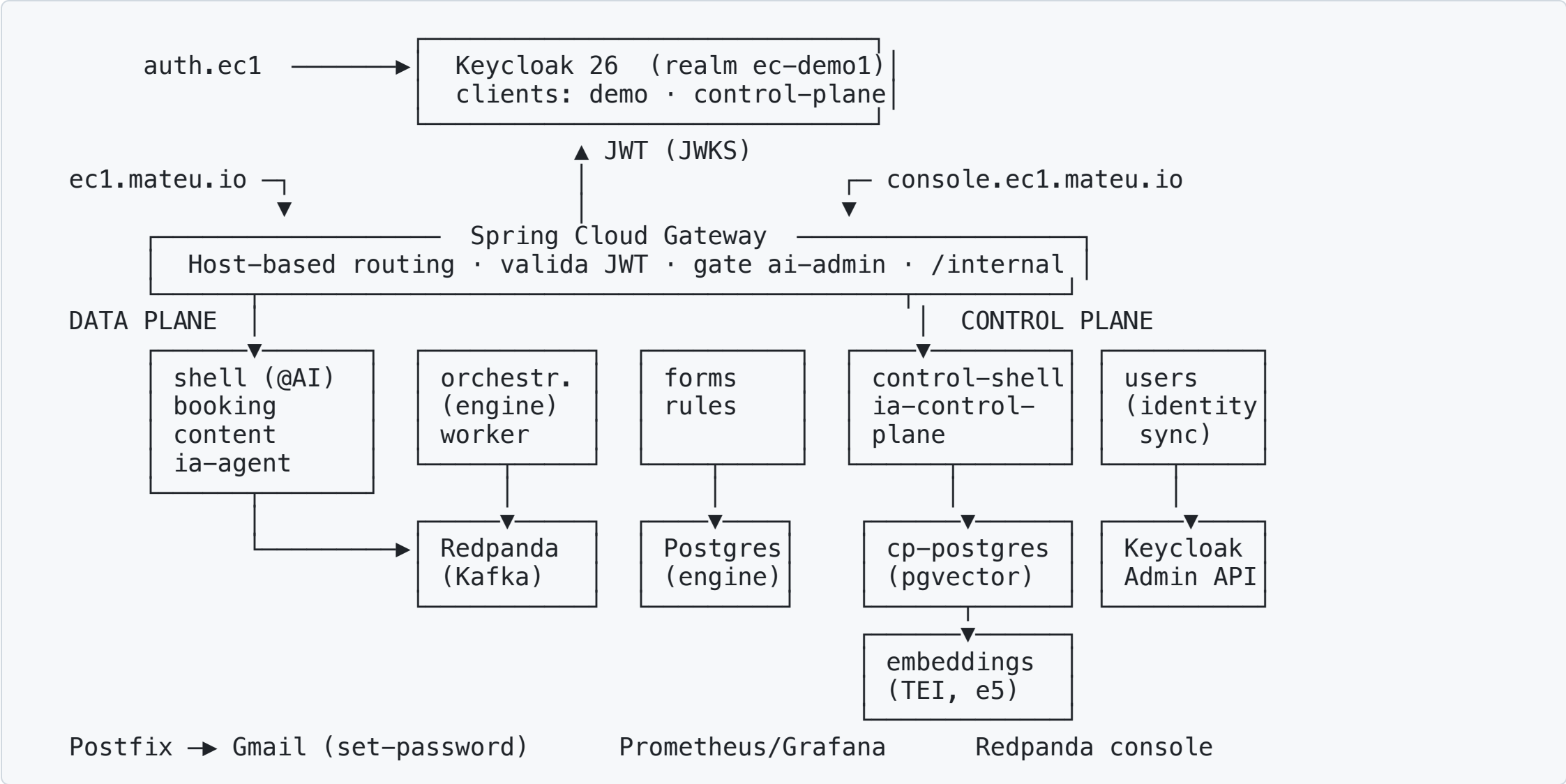
Dos consolas, un gateway, un principio: **separar ejecutar de gobernar**.

Los dos planos

	Data plane — <code>ec1.mateu.io</code>	Control plane — <code>console.ec1.mateu.io</code>
Para qué	Usar el producto	Gobernar el producto
Quién	Usuarios de negocio	<code>ai-admin</code>
Contiene	Motor de workflow, forms, reservas, contenido, agente IA	Catálogos IA (LLM/MCP/RAG/agente), presupuestos, enrutado, identidad
Naturaleza	Runtime, alta frecuencia, datos de negocio	Configuración, gobernada, versionada en git
Cambia	Cada segundo (procesos, tareas)	Rara vez, con revisión

Dos **clientes Keycloak** distintos ⇒ un token de una consola no vale en la otra.

Mapa de componentes



Data plane — el negocio que corre

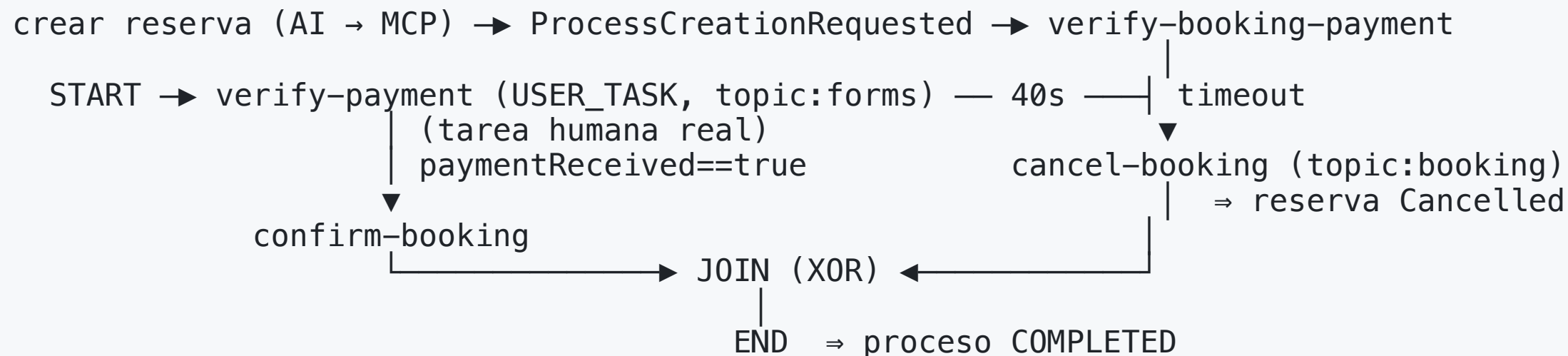
- **Motor de workflow** (`orchestrator` , EventConductor 2.10.3) — ejecuta procesos, pasos, sagas.
- **Forms** (2.10.2) — tareas humanas; **rules** (2.10.2) — decisiones declarativas.
- **worker** — worker de prueba que responde tareas en carga.
- **booking / content** — servicios de negocio (hexagonal/DDD).
- **shell** — la consola RIU; una sola página que compone las UIs remotas de cada servicio.
- **ia-agent** — el chat: panel `@AI` del shell, habla MCP con los servicios.
- **Redpanda** (Kafka) + **Postgres** — bus de eventos y estado.

Control plane — gobernar la IA y el acceso

- **ia-control-plane** — catálogos versionados: LLM · MCP · RAG · Agente, más presupuestos de tokens y reglas de enrutado.
- **users** — administración de identidad; sincroniza a Keycloak (crear/editar/borrar).
- **cp-postgres** (pgvector) — estado del control plane y vectores del RAG.
- **embeddings** (TEI, multilingual-e5-small) — embeddings locales, sin segundo proveedor.
- **control-shell** — la consola de administración: menús IA y Usuarios.

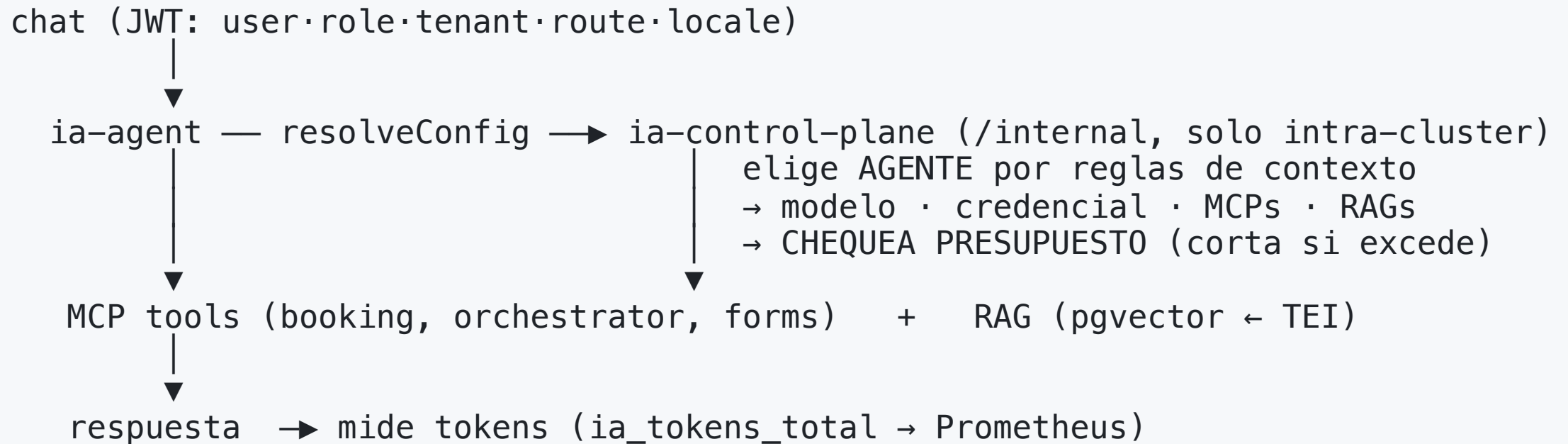
Todo detrás del rol **ai-admin**, en un host propio.

Flujo 1 — Reserva gobernada por workflow (*la demo en vivo*)



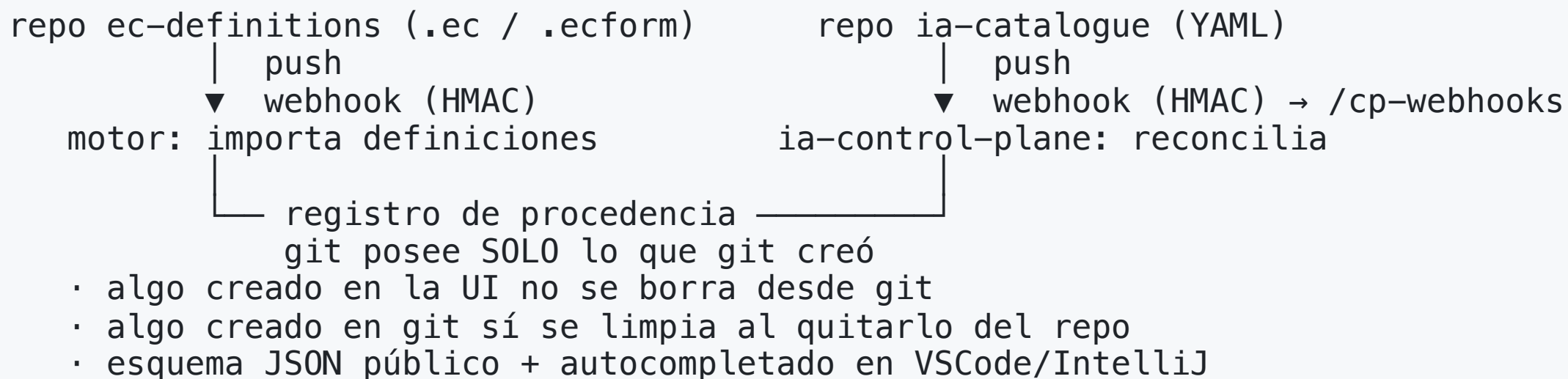
Verificado end-to-end: reserva **H9VXSM** → **timeout a los 40 s clavados** → cancelada → COMPLETED.

Flujo 2 — El agente IA, gobernado por contexto



El control plane decide **qué agente, con qué llaves y qué presupuesto** — el data plane solo ejecuta.

Flujo 3 — GitOps: la configuración vive en git



Configuración como código, sin que git pise lo que edita un operador.

Flujo 4 — Identidad federada, sin secretos en el repo

```
users (CRUD) → evento de dominio → outbox transaccional → Keycloak Admin API
                                                         · crea / edita / borra
                                                         · email set-password
                                                         |
                                                         Postfix → Gmail (relay autenticado)
```

- **Outbox transaccional:** el cambio y su publicación se confirman juntos, o ninguno.
- **Postfix** aísla el secreto (App Password + TLS) — el realm en git no lleva credenciales.
- Roles/scopes se crean en `users` y el **gateway enriquece el token**.

Modelo de seguridad

- **Dos clientes Keycloak** = dos audiencias. El control plane exige `ai-admin`; el data plane, roles de negocio.
- **Gateway**: enrutado por *host*, valida JWT contra JWKS (no depende del arranque de Keycloak).
- `/internal` **no se enruta**: la config resuelta del agente (con su API key en claro) solo se alcanza dentro del cluster.
- **Webhooks** (git, GitOps) autentican por **HMAC** sobre el cuerpo — GitHub no puede llevar token.

Infraestructura

- **Kubernetes** en CloudFleet (Hetzner). Nodo `cx43` (amd64); imágenes single-arch con `nodeSelector`.
- **Postgres** — motor + una BD por servicio. **pgvector** — control plane + RAG.
- **Redpanda** — topics `upstream` / `downstream`. **TEI** — embeddings locales.
- **Prometheus + Grafana** — `ia_tokens_total`, paneles de consumo; **Redpanda console** para el bus.
- **Anthropic** como LLM (Claude); embeddings **locales** para no atar un segundo proveedor.

Principios (y hoja de ruta)

- **Separar ejecutar de gobernar** guía todo el montaje: config + identidad + IA en el control plane; runtime + datos de negocio en el data plane.
- **Definiciones de workflow** ya son *de control* por forma (git, esquema, procedencia) → candidatas a gestionarse desde la consola de control; la **ejecución** se queda en el data plane.
- **Analíticas a escala de producción**: sustituir el agregado on-demand por un **read model actualizado offline** (CQRS) — resúmenes por día que se leen en milisegundos, no un scan del histórico.

Gracias

Data plane ejecuta · **Control plane** gobierna · **un gateway** los guarda

Demo: reserva → tarea humana → *timeout* 40 s → cancelación → COMPLETED